PQC-LINK: Progress Report

Phase 1 — a hybrid post-quantum TLS link between two computers

RSCL senior design team

October 17, 2026

Abstract

Following the advisor's guidance, we built and tested the complete secure communication link on computers we already own, before touching any Raspberry Pi or Pico hardware. The link uses OpenSSL 3.5.9 and TLS 1.3 and compares classical key exchange (`X25519`, configuration A) with hybrid post-quantum key exchange (`X25519MLKEM768`, configuration B). This report shows the architecture, the security design, how the pieces were verified, and a first set of performance measurements. The measurements are *preliminary*: they were taken on a development computer, not on the target hardware, and they are not gate evidence.

1 Where the project stands

The advisor asked for a working implementation with verified encryption, decryption, communication and performance measurements before any hardware migration. Table 1 summarises what exists today.

Table 1: Phase 1 status. "Tested" means automated tests pass on a development computer.

Area	What exists	State
OpenSSL 3.5.9	Pinned build, reproducible team guide, hybrid group confirmed	Done
TLS link	Client and server, groups A and B, one cipher suite, verified peer	Tested
Wire format	Numbered RECORD and ACK messages, length-checked	Tested
Failure cases	Downgrade, wrong hostname, untrusted CA, tampered record, timeout	Tested
Benchmark	Interleaved A/B handshake harness, summary tool, runner script	Tested
Durable store, dedup	Server commit step is a stub; an ACK is not yet "stored"	Next
Queue, outage, loss tests	Not started	Planned
Pico, IMU, OLED, Wi-Fi	Phase 2, after the advisor reviews Phase 1	Deferred

2 System architecture

Figure 1 shows the full system from the project scope. In Phase 1 only the two computers in the middle exist: a client that plays Pi A and a server that plays Pi B. The sensor, display and Pico boards are Phase 2 and are not simulated.

Figure 2 shows how the software is layered. The programs never call OpenSSL directly: one small module, `tls_link`, fixes every setting the experiment must keep constant, so a mistake cannot quietly change what is measured.

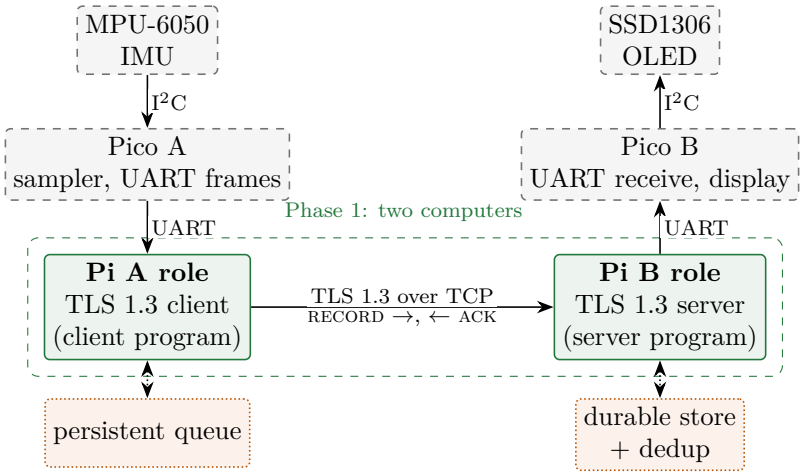


Figure 1: System architecture. Solid green: built and tested in Phase 1. Dotted orange: next. Dashed grey: Phase 2 hardware.

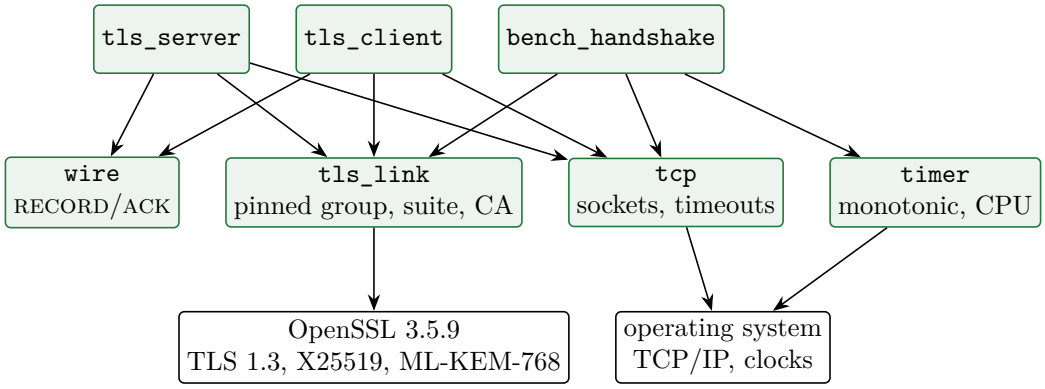


Figure 2: Software layers. Arrows point from a user to what it uses.

3 Security design

3.1 What is fixed

Configurations A and B differ in exactly one setting, the key-exchange group. Everything else in Table 2 is identical, as required for a fair comparison.

Table 2: Settings enforced by `tls_link` on both the client and the server.

Setting	Configuration A (classical)	Configuration B (hybrid)
TLS version	1.3 only (min = max)	1.3 only (min = max)
Key-exchange group	`X25519` only	`X25519MLKEM768` only
Cipher suite	`TLS_AES_128_GCM_SHA256`	`TLS_AES_128_GCM_SHA256`
Session tickets, resumption	Off: every handshake is a full one	Off
Server certificate	Pinned demo CA, hostname checked	Pinned demo CA, hostname checked
Fallback to another group	Never	Never

3.2 Fail closed, with two independent checks

Figure 3 shows what a client does. A peer that cannot meet the policy ends the connection; there is no retry with a weaker group and no way to switch verification off. After the handshake, the negotiated version, group and cipher suite are checked a second time against the pinned values, and

a client confirms that a verified peer certificate exists. Removing one check by mistake therefore does not open a hole.

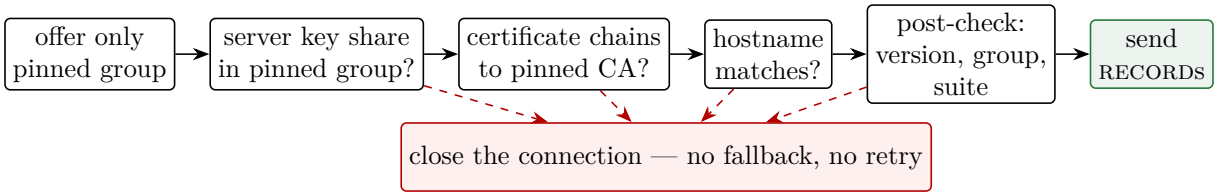


Figure 3: Client-side checks, in order. Any failure ends the connection.

3.3 What the handshake carries

Figure 4 shows the TLS 1.3 handshake. The only visible difference between the two configurations is the size of the key shares: the hybrid client sends an ML-KEM-768 encapsulation key (1184 bytes, FIPS 203) next to its X25519 key, and the server answers with an ML-KEM-768 ciphertext (1088 bytes).

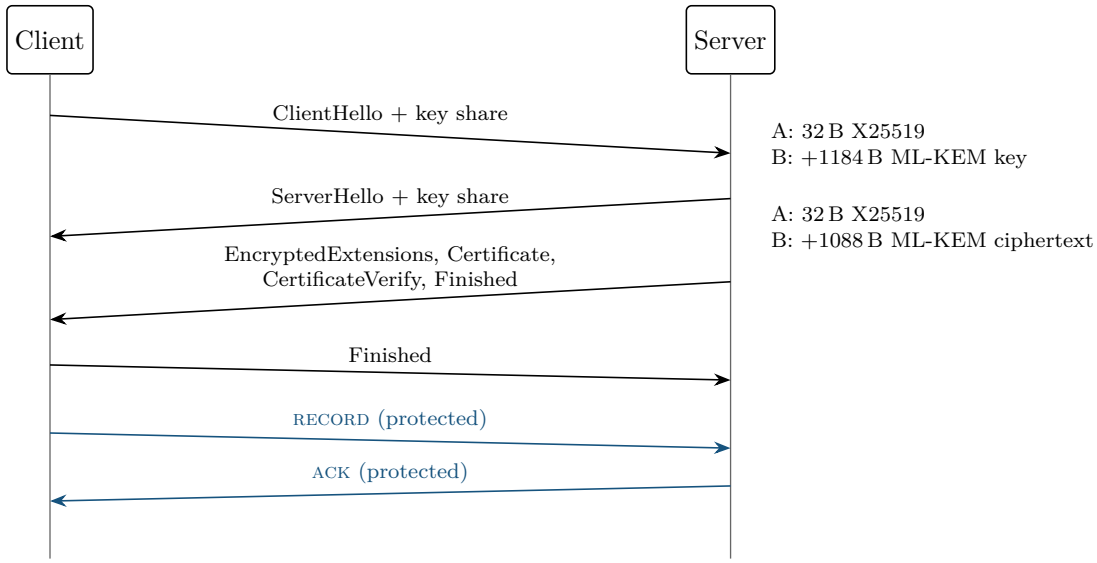


Figure 4: TLS 1.3 handshake and the first application messages. Everything after ServerHello is encrypted.

3.4 Message format

The application protocol is deliberately small. Every message is *type, length, payload*, big-endian, and a receiver checks the announced length against the exact size for that type before it reads the payload. A RECORD is 36 bytes on the wire and an ACK is 12 (Figure 5). In Phase 1 the six IMU values are zero, since no sensor exists yet.

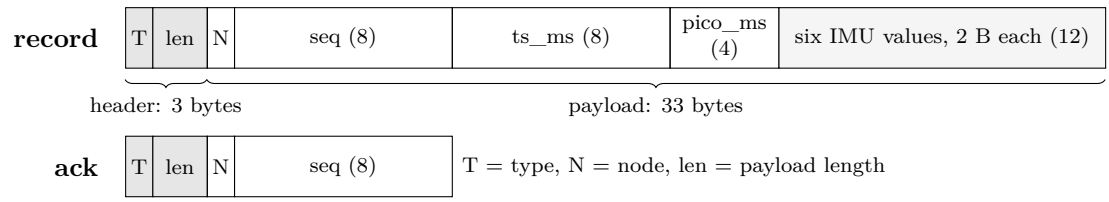


Figure 5: Wire format. Grey = header. Widths are proportional to bytes. Sequence numbers start at 1; a RECORD with sequence 0 is rejected.

4 How it was verified

Table 3 maps each requirement to an automated check. All 10 automated tests (two of them only generate demo certificates) pass on the development computer. The failure cases run as library tests over a socket pair; the group and certificate cases also run as real server and client processes over TCP.

Table 3: Requirements and how they are checked. "Dev" = development computer only.

Test	Requirement	Automated check	State
T02	Log the negotiated group	Group and suite read back after the handshake, both configs	Dev
T03	Wrong hostname or untrusted CA fails closed	Client rejects with a verification error; server sees no session	Dev
T04	Classical-only vs hybrid-only fails	Both directions rejected, no silent downgrade	Dev
T09	Tampered TLS ciphertext rejected	Corrupted record is an I/O error, never delivered	Dev
T10	100+ handshakes, A vs B	Harness built and exercised; see Section 5	Prelim.
T05	Replayed record counted once	Needs the durable store	Next
T06–T08	Outage, restart, 10%/30% loss	Needs queue and loss emulation	Planned
T01	Tilt changes the display	Needs Pico, IMU, OLED (Phase 2)	Deferred

5 Performance measurement

5.1 Method

Figure 6 shows one measured attempt. Configurations A and B are *interleaved* (A, B, A, B, ...) so that slow changes in load or temperature affect both equally. Every attempt is a fresh TCP connection and a full handshake. Nothing is written or printed inside the timed span; rows are kept in memory and saved afterwards. Failed attempts go to a separate file and are never mixed into the statistics. The first five successes per configuration are warm-up and are excluded.

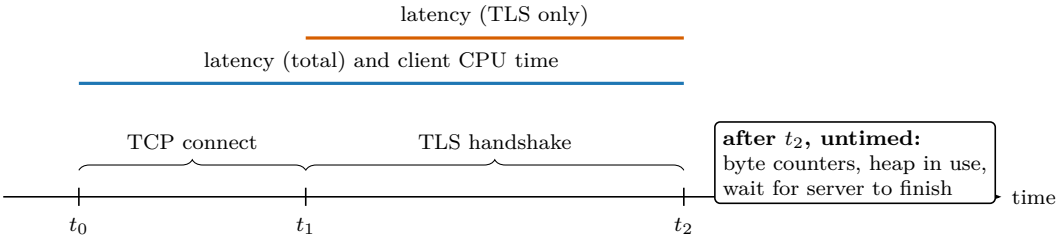


Figure 6: One measured handshake attempt.

Per attempt the client records: total latency (t_0 to t_2) and TLS-only latency (t_1 to t_2); client CPU time over the same span; handshake bytes sent and received, counted by OpenSSL on the socket (the TLS byte stream, without TCP/IP headers); and the heap held by the open connection. The transient memory peak inside the handshake is not captured per attempt. Server-side CPU and memory are not measured yet.

5.2 Conditions

- Platform: Intel(R) Core(TM) Ultra 9 285H, WSL2; client and both servers on one machine over loopback.
- OpenSSL 3.5.9; Release build; git commits 22d09de, 26debf4 (working tree modified: yes).
- 5 separate sessions of 100 measured handshakes per configuration (500 per configuration in total), no failures. Measured on 2026-10-17.
- Negotiated groups seen: `x25519` (A) and `x25519mlkem768` (B).

Preliminary: one development computer, loopback network, no packet loss. These numbers show that the harness works and what the size and cost of the hybrid handshake look like here. They are not results for the target hardware and must not be quoted as such.

5.3 Handshake size

The byte counts are deterministic, so they are the most solid result so far. Figure 7 shows that the hybrid handshake sends 1176 bytes more and receives 1088 bytes more than the classical one. These differences are exactly the ML-KEM-768 encapsulation key and ciphertext that the handshake carries.

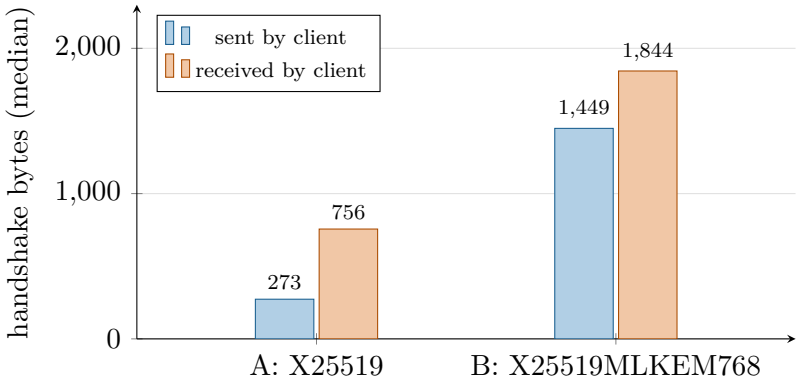


Figure 7: Handshake bytes on the TLS stream, median over 500 handshakes per configuration.

5.4 Latency and CPU time

Figure 8 shows the spread of the 500 handshakes per configuration. The hybrid handshake is slower, with a median total latency of 0.559 ms against 0.469 ms (95th percentiles 0.736 and 0.620 ms). Client CPU time is 0.359 ms against 0.308 ms. The open connection holds about 26.8 KiB of heap in configuration B and 19.1 KiB in configuration A.

5.5 Repeatability

A single session can mislead, so Figure 9 shows each of the 5 sessions separately. The absolute values move from session to session, but configuration B is slower than A in every session. The B-to-A ratio of the median latency stays between 1.16 and 1.24 (median 1.18). That is a consistent trend, but on one machine with a loopback network it says little about how a Raspberry Pi over Wi-Fi will behave.

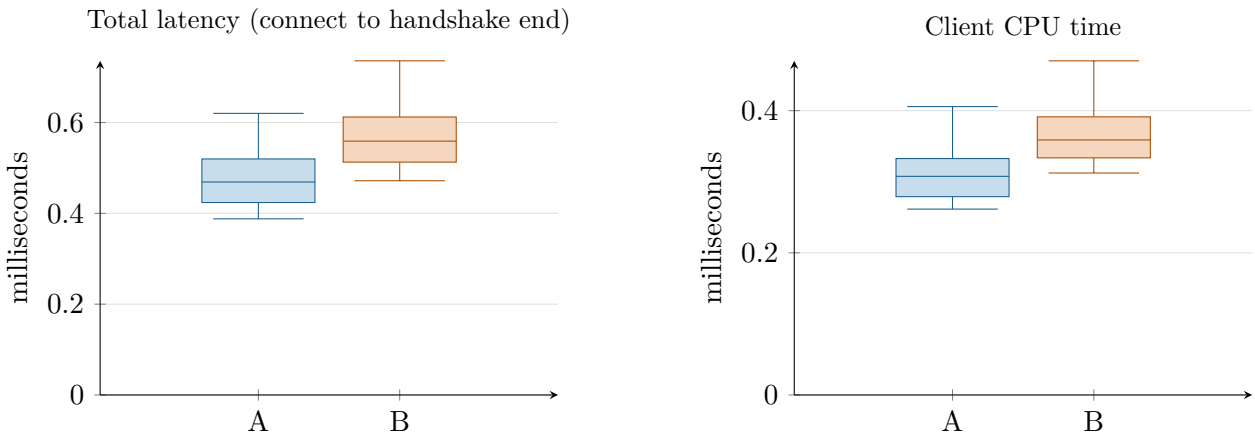


Figure 8: Distribution over all 500 handshakes per configuration. Box: 25th to 75th percentile with the median line; whiskers: 5th and 95th percentile. A = classical, B = hybrid. Preliminary, development computer.

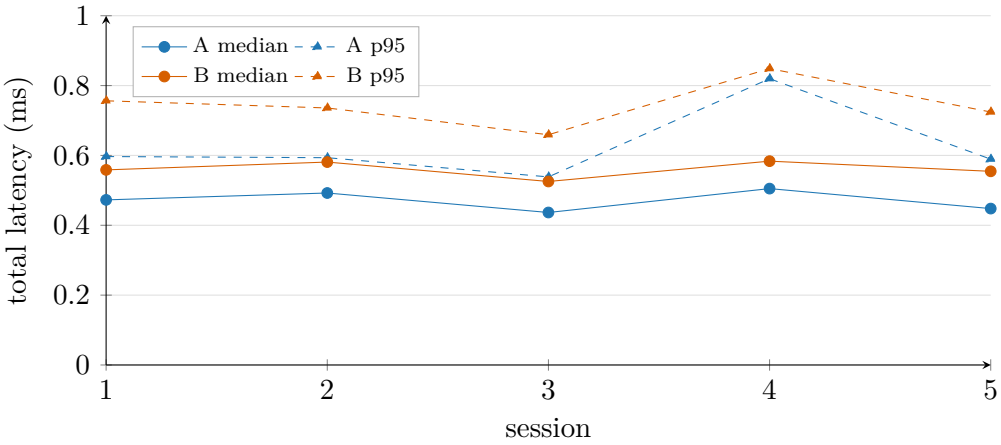


Figure 9: Median and 95th-percentile total latency in each session (100 handshakes per configuration per session).

6 Claims and limits

- **Hybrid key establishment is not post-quantum authentication.** The server certificate is a classical ECDSA P-256 demo certificate. Only the key exchange is hybrid.
- This is an educational link. It is not flight-approved, not space-qualified and not endorsed by any agency.
- Results come from a development computer over loopback. Nothing here is gate evidence; that requires the target hardware, recorded board identifiers and the instructor's initials.
- The server's acknowledgment is not yet tied to durable storage, so it does not yet mean "committed".
- Handshake bytes exclude TCP/IP headers; a packet capture is still needed for those totals.
- Server-side CPU and memory are not measured yet.

7 Roadmap

Figure 10 places the work against the ten build gates. A software part being implemented does not mean the gate is passed: each gate still needs its hardware evidence.

Next steps.

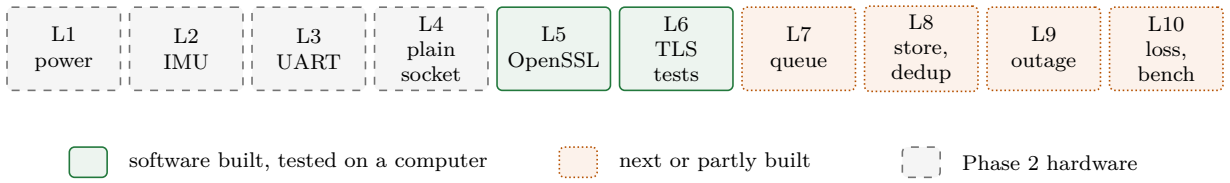


Figure 10: Build gates L1–L10. Gate L10 (benchmark) is partly built: the handshake harness exists, the loss runs do not.

1. Durable store with duplicate detection, so an acknowledgment means the record is stored (T05).
2. Persistent queue, outage recovery and packet-loss tests (T06–T08).
3. Repeated benchmark runs with server-side CPU and memory, and packet captures for byte totals.
4. Review with the advisor, then plan the Pico W migration.

Questions for the advisor. How should packet loss be emulated (`tc netem` on the computers or at the access point)? May the Pi services stay in C? May we reuse the existing CSV logger for evidence files? Is a second computer needed for the final Phase 1 measurements?